

仕様書の進捗・在庫・計画在庫計算に関する調査報告

ご依頼の通り、仕様書と実コードを照合し、計算ロジック、開始日、および懸念点をまとめました。

1. 計算ロジックと開始日の整合性確認

仕様書（[在庫計算仕様書.md](#)）と実装コード（[inventory_calculator.py](#), [progress_calculator.py](#)）の整合性を確認しました。

項目	コード上の関数名	計算開始日 (Anchor Date)	仕様書との整合性
実在庫	<code>recalculate_stock_qty</code>	前々営業日 (<code>day_before_yesterday</code>)	☒ 一致
計画在庫	<code>recalculate_planned_stock_qty</code>	今日 - (最大LT + 1)営業日	☒ 一致
進捗	<code>recalculate_progress_qty</code>	今日 - (最大LT + 1)営業日	☒ 一致

計算式の詳細確認

- **実在庫:** 前日在庫 + 実績 - 出庫(親実績+仕損) + 調整
 - コード: `stock_qty = prev_stock + actual_total - actual_shipment + scrap_adjust_total`
 - 仕様通り。
- **計画在庫:**
 - 過去(<=Today): 前日計画 + 実績 - 出庫(親実績+仕損)
 - 未来(>Today): 前日計画 + 計画 - 出庫(親計画)
 - コード: 時制による分岐ロジックを確認。仕様通り。
- **進捗:** 前日進捗 + 実績 - 需要(LTシフト済) + 調整 - 仕損
 - コード: `progress_qty = prev_progress + actual_total - progress_shipment + adjust_total + scrap_adjust_total`
 - 仕様通り。

2. 懸念点：棚卸結果の上書きリスク

最大の懸念点は、「**棚卸初期化**」で設定した直後に「**自動再計算**」が走ると、棚卸結果が上書きされて消える可能性がある点です。

発生メカニズム

1. **棚卸初期化:**

- `initialize_stocktake` を実行し、例えば「昨日」時点の在庫・進捗を正しい値（実地棚卸値）で上書き保存します。

2. 自動再計算:

- 翌日（または同日）、`recalculate_inventory` が実行されます。
- この関数は、**計算開始日（最大LT分、例えば10日前）**まで遡って計算を開始します。
- 計算ロジックは **前日値 + 入庫 - 出庫** の積み上げ計算です。
- 途中で「棚卸確定値」があってもそれを参照せず、**10日前からの積み上げ計算で「昨日」の値を再計算して上書き**してしまいます。

具体例

- **設定:** 最大リードタイム = 5日
- **操作:** 2月20日に「2月19日（昨日）」付で棚卸結果を投入。在庫=100個で確定。
- **再計算:** 2月21日に定期ジョブ等で再計算が走る。
 - 開始日 = 2月21日 - (5+1)日 = **2月15日**
 - 計算: 2月15日から積み上げ計算を開始。
 - 2月19日の計算: **2月18日在庫 + 19日入荷 - 19日出荷 = 計算値（例: 98個）**
 - **結果:** 棚卸で入れた「100個」が、計算値「98個」で上書きされる。

推奨される対策

再計算ロジック（`inventory_calculator.py` / `progress_calculator.py`）において、「**棚卸確定フラグ**」または「**実績確定ロック**」のような仕組みを導入するか、計算開始日を「直近の棚卸日」以降にする動的な制御が必要です。

現状のコードには、`recalculate_stock_qty` 等の中で「この日は棚卸日だから計算せずにその値を採用する」という分岐が見当たりません。運用回避としては、「棚卸直後に必ずシステム在庫と実在庫の差分を `adjust_qty` に入力する」という方法のみになりますが、`initialize_stocktake` 機能の存在意義（一括リセット）と矛盾します。